

Performance of Metaheuristics

Introduction

Contrairement aux algorithmes classiques (ex: tri), une analyse de complexité traditionnelle n'est tout simplement pas possible avec les métaheuristiques. Elles ne fournissent pas un temps d'exécution prévisible. Pour certaines, nous avons des résultats de convergence, mais malheureusement limités à des cas particuliers d'intérêt pratique limité.

Par conséquent, les analyses de performance des métaheuristiques sont principalement empiriques et requièrent une approche statistique, car les métaheuristiques ne sont pas déterministes (elles comportent toutes de multiples composantes stochastiques).

Challenges in Performance Analysis

Une liste non exhaustive des défis inclut :

- **Stochasticité et variance entre les exécutions**
- **Manque d'indicateurs de performance universels**
- **Dépendance aux paramètres de guidage**
- **Dépendance au problème et manque de généralité**
- **Difficulté à mesurer la convergence et l'effort de calcul**
- **Évolutivité et sensibilité à la dimensionalité**
- **Interactions complexes entre exploration et exploitation**

Example: Traveling Salesman Problem (TSP) with Simulated Annealing

Idée derrière l'exemple

Cet exemple illustre comment mesurer l'effort de calcul et la qualité de la solution pour une métaheuristique (Recuit Simulé) appliquée au TSP. Pour des problèmes de taille croissante, on observe un changement de régime de complexité.

Guided Parameters du Recuit Simulé

- **Température initiale** : Contrôle la probabilité d'accepter des solutions moins bonnes en début de recherche.
- **Calendrier de refroidissement (cooling schedule)** : Fonction qui réduit la température au cours du temps (ex: décroissance géométrique).
- **Nombre d'itérations par palier de température** : Définit l'équilibre thermique à chaque température.
- **Température finale** : Critère d'arrêt lié à la température.
- **Critère d'arrêt alternatif** : Nombre maximum d'itérations ou absence d'amélioration sur un certain nombre d'itérations.

Résultats synthétiques

Pour N villes dans un domaine 2D, l'effort de calcul $T(N)$, mesuré en nombre d'évaluations de la fonction objectif, croît approximativement comme $T(N) = 7100 \times N^{1.14}$ pour N entre 20 et

2000. Pour des problèmes plus grands (N entre 5000 et 50000), le taux de croissance est plus élevé, indiquant un changement de régime.

La qualité de la solution (erreur relative par rapport à l'optimum connu) s'améliore avec l'augmentation de l'effort de calcul, et la probabilité de succès (trouver une solution dans une marge d'erreur ϵ) augmente également.

Example: NK-landscape

Idée derrière l'exemple

Le modèle NK est un problème de benchmark synthétique où l'on peut contrôler la rugosité du paysage de fitness via les paramètres N (nombre de gènes) et K (épistasie). Il permet d'étudier la performance d'algorithmes génétiques sur des paysages de complexité variable.

Guided Parameters de l'Algorithme Génétique

- **Taille de la population (number of individuals)** : Influence la diversité génétique et l'exploration.
- **Opérateurs de sélection** : Méthode pour choisir les parents (ex: roulette, tournoi).
- **Taux de croisement (crossover rate)** : Probabilité d'appliquer l'opérateur de croisement.
- **Taux de mutation (mutation rate)** : Probabilité de modifier un gène.
- **Critère d'arrêt** : Nombre maximum de générations ou stagnation de la fitness.

Résultats synthétiques

Pour des problèmes avec N entre 8 et 20 et $K = 5$, la complexité moyenne en temps (sur 500 problèmes générés aléatoirement) augmente avec N . La fraction de problèmes pour lesquels l'optimum global n'est pas trouvé dans le temps alloué (80 générations) augmente également avec N , et augmenter le nombre de générations n'améliore pas les résultats pour ces instances difficiles.

La probabilité de succès (trouver une solution à ϵ près de l'optimum) et l'erreur relative moyenne dépendent fortement de l'effort de calcul (ajusté en variant le critère d'arrêt basé sur un plateau de fitness) et de la valeur de K (plus K est élevé, plus le paysage est complexe).

Performance Evaluation: General Concepts

Les métaheuristiques sont testées sur des problèmes de référence (benchmarks). Ces benchmarks peuvent être des problèmes réels ou synthétiques. Les problèmes synthétiques (comme les NK-landscapes) permettent de construire des espaces de recherche avec des niveaux de complexité croissants.

Performance Metrics

Les métriques clés pour évaluer la performance des métaheuristiques sont :

- **Effort de calcul**
- **Qualité de la solution**
- **Évolutivité** (croissance du temps de calcul avec la taille du problème, en espérant qu'elle ne soit pas exponentielle)

Pour déterminer ces métriques, une moyenne sur $\mathcal{N} \in [100, 1000]$ exécutions est typiquement nécessaire.

Computational Effort

L'effort de calcul peut être mesuré simplement par le temps écoulé (wall-clock time). C'est facile à obtenir mais dépendant de la machine.

Alternativement, on peut compter le nombre total d'opérations (ou d'évaluations de la fonction objectif). Cela permet des comparaisons absolues entre métaheuristiques et machines, mais n'est pas applicable si la fonction objectif n'est pas mathématiquement connue (problèmes réels).

Quality of Solution

On estime statistiquement la probabilité de trouver une solution en fonction de l'effort de calcul. Cette probabilité est donnée par le taux de succès (nombre de fois où la solution a été trouvée sur $\mathcal{N}$ exécutions).

Cette approche nécessite de connaître l'optimum global, ce qui est souvent le cas avec les benchmarks. Sinon, on peut utiliser la meilleure solution connue, ou estimer la probabilité de trouver une solution dont la fitness est à $x\%$ de la fitness optimale.

Statistical Comparison

Pour comparer deux métaheuristiques appliquées au même problème, il est impératif d'utiliser des tests statistiques (ex: Kolmogorov-Smirnov) pour s'assurer que la différence de performance obtenue est statistiquement significative.

La **robustesse** est une propriété critique : il s'agit de l'efficacité générale de l'approche sur une large classe de problèmes plutôt que d'être excellente pour seulement 1 ou 2 problèmes spécifiques.

The “No Free Lunch” Theorem (NFL)

Énoncé verbal

Une métaheuristique, quelle qu'elle soit, ne peut pas surpasser une autre sur l'ensemble complet de tous les problèmes. Pour toute mesure de performance, aucun algorithme de recherche unique n'est meilleur qu'un autre lorsqu'ils sont comparés sur toutes les fonctions discrètes possibles. Si une méthode A surpasse une méthode B pour une classe de problèmes donnée, alors il existe une autre classe de problèmes pour laquelle B surpasse A .

Hypothèses

On considère un espace de recherche S de dimension finie, de taille $|S|$. On considère des fonctions de fitness $f : S \rightarrow Y$ où Y est un sous-ensemble fini de $\mathbb{R}$. Tous les problèmes possibles dans S sont spécifiés par une fitness parmi les $|Y|^{|S|}$ possibles. On considère en outre que l'effort de calcul est m , de sorte que la trajectoire exploratoire est une séquence d_m qui contient m couples $(x_i, f(x_i))$, pour $i = 1, \dots, m$.

Énoncé mathématique

Soit $P(d_m \mid f, m, A)$ la probabilité que la séquence d_m générée par l'algorithme A contienne l'optimum de f . Le résultat principal du théorème NFL s'écrit :

$$\sum_f P(d_m \mid f, m, A_1) = \sum_f P(d_m \mid f, m, A_2)$$

Ainsi, la performance moyenne de **TOUT** algorithme de recherche sur **TOUS** les problèmes est identique.

Implications pratiques

En pratique, tous les problèmes ne sont pas équiprobables et les problèmes “pathologiques” sont moins fréquents. Par conséquent, on peut espérer que pour certaines classes de problèmes, certaines métaheuristiques surpassent les autres. Le théorème NFL nous rappelle qu'il n'existe pas de méthode universellement meilleure et qu'il est important de choisir l'algorithme adapté au problème.